

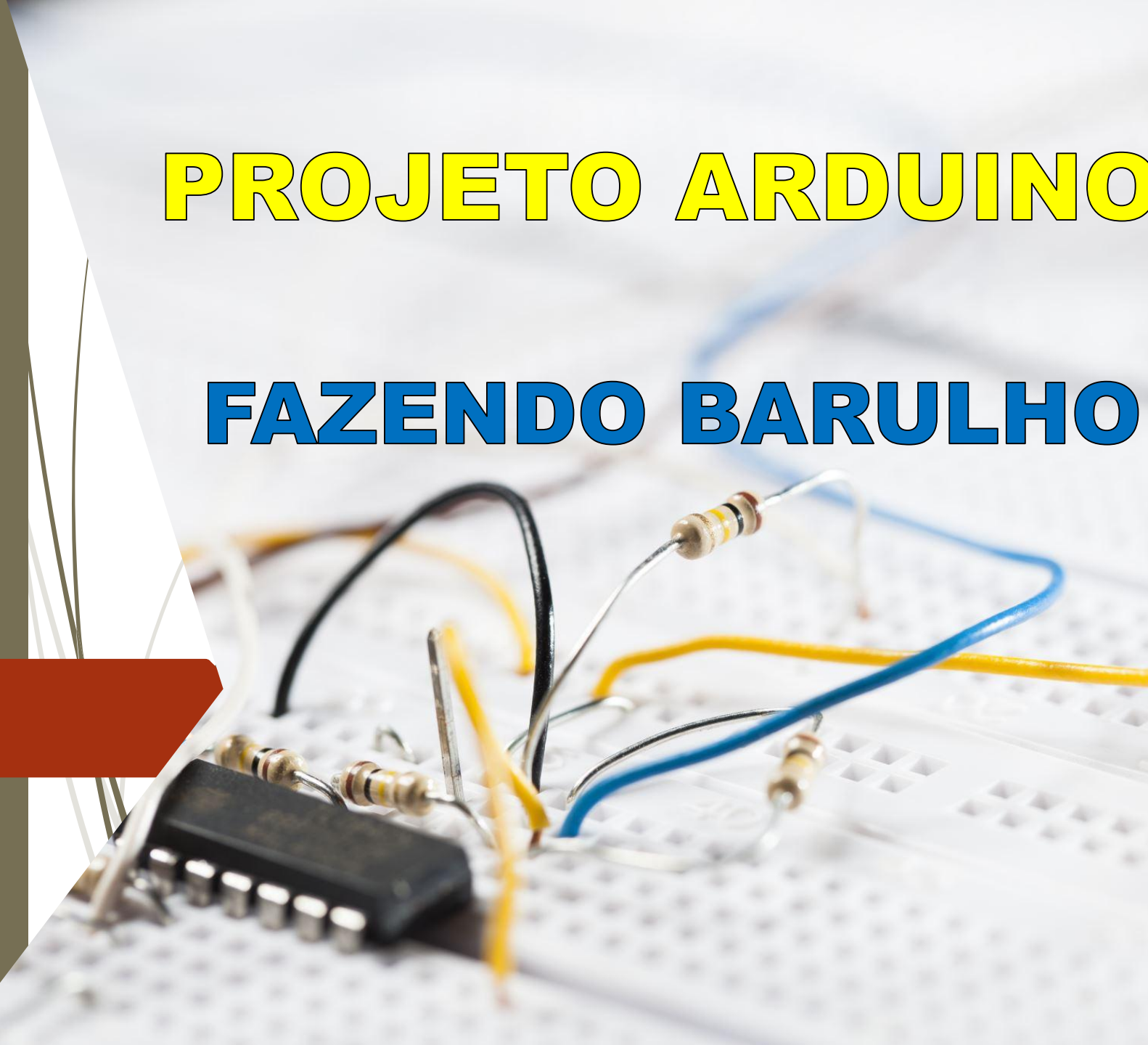
PROJETO ARDUINO – Parte 02

Prof. Sergio Luiz da Silveira



PROJETO ARDUINO – Parte 02

FAZENDO BARULHO



INTRODUÇÃO

Lado a lado aos **sinalizadores luminosos**, os **sinais sonoros** são muito utilizados em uma grande gama de projetos.

Podendo ser encontrado em máquinas, interfaces de eletrodomésticos, sistemas de controle de acesso, etc, o buzzer será o objeto de estudo deste capítulo.

COMO FUNCIONA O BUZZER

Buzzers podem ser divertidos e úteis em circuitos eletrônicos.

O **buzzer** consiste em um invólucro exterior com dois pinos para **ligá-lo ao 5V e GND**.

Dentro desse invólucro existe um elemento piezoelétrico que consiste em um disco cerâmico central rodeado por um disco de vibração de metal, normalmente feito de bronze.



COMO FUNCIONA O BUZZER

Quando aplicamos uma corrente ao buzzer, o disco de cerâmica é contraído ou expandido. Isso é o que gera o som que ouvimos.

Variando a frequência do sinal no buzzer, **a velocidade das vibrações é alterada** e **modificam o passo do som resultante**.

BUZZER PASSIVO e BUZZER ATIVO

Em geral é possível encontrar dois tipos de buzzer: o **ATIVO** e o **PASSIVO**. De construção mais complexa, o **buzzer ativo** é o mais fácil de utilizar.

Ele já possui um circuito oscilador interno que reproduz um som sempre da mesma frequência quando é energizado.

BUZZER PASSIVO e BUZZER ATIVO

Já o **BUZZER PASSIVO** é mais simples, apenas um transdutor similar a um pequeno autofalante.



Sendo assim, o microcontrolador deve gerar a frequência para que ele emita um som.

HARDWARE

O primeiro passo que tomaremos é **fazer o controle simples do buzzer**. A ligação deste componente é similar à ligação de um **LED**.

Entretanto, não é necessário utilizar um resistor para limitar a corrente dele, você deve-se apenas ficar atento à sua polaridade.

HARDWARE

Observe que na parte de cima de seu buzzer existe uma marcação com o sinal de **positivo** que indica o terminal positivo do mini autofalante.



Você deve então obedecer esta polaridade na hora de ligá-lo.



PRATICANDO

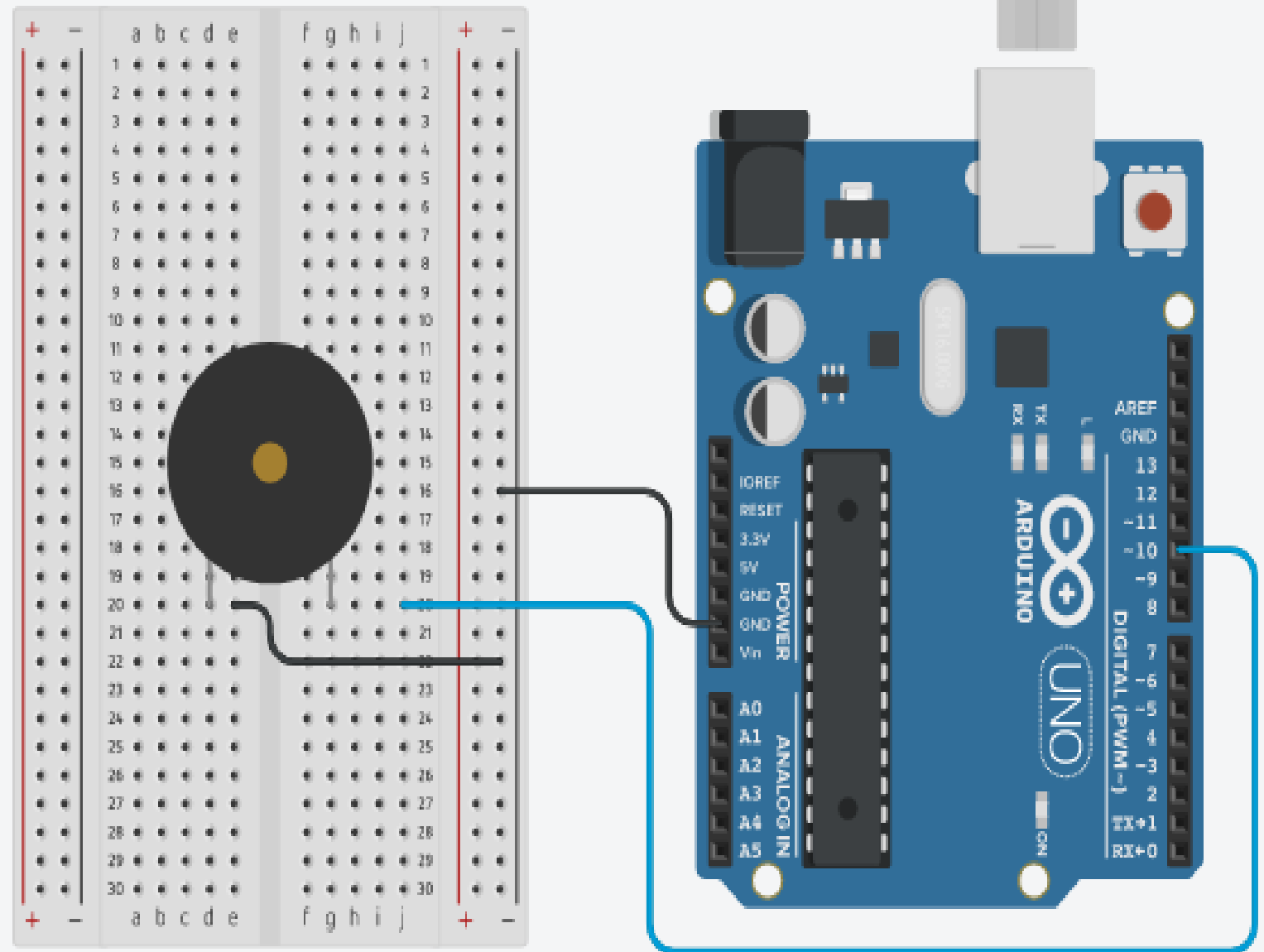
LISTA DE MATERIAIS

Você precisará dos seguintes materiais para o desenvolvimento dessa atividade:

- 1x **Placa Arduino + Cabo USB**
- 1x **Buzzer Passivo 5V**
- 2x **Chave Momentânea(PushButton)**
- 1x **Protoboard**
- **Jumpers**

FAZENDO BARULHO SIMPLES

Veja um exemplo simples da ligação do **BUZZER** à placa Arduino:



FAZENDO BARULHO SIMPLES

Desenvolvendo o **CÓDIGO** para testar. – **PISCA-PISCA SONORO**

```
1. /*****
2. * Projeto 2 - Fazendo Barulho 1
3. * Pisca-pisca sonoro.
4. *****/
5.
6. int frequencia = 2000;
7.
8. void setup() {
9.     pinMode(10, OUTPUT); // configura o pino com
    o buzzer como saída
10. }
11.
```

CONTINUA...

FAZENDO BARULHO SIMPLES

...CONTINUAÇÃO

```
12. void loop() {  
13.   tone(10, frequencia); // gera frequencia de  
    2000Hz no buzzer  
14.   delay(1000);  
15.   noTone(10); // para frequencia no buzzer  
16.   delay(1000);  
17. }
```

FIM!

ENTENDENDO O CÓDIGO

Consegue perceber a semelhança desse código com o código **Blink**?

Acionamos o buzzer por um segundo e desativamos por mais um segundo em um **LOOP**, resultando assim em um "pisca-pisca sonoro".

Veja que, ao contrário de como fizemos com o **LED**, não utilizamos a função **digitalWrite** (isso até funcionaria se nosso buzzer fosse ativo, ou seja, se ele tivesse um oscilador interno).

ENTENDENDO O CÓDIGO

Usamos a função **tone**, pois o nosso buzzer é passivo e precisamos gerar uma frequência para que ele emita algum som.

Para fazer o acionamento utilizamos uma instrução **tone(10, frequencia)**.

ENTENDENDO O CÓDIGO

Sua utilização é bem simples: devemos definir, nessa ordem, em qual **pino** e em qual **frequência** iremos gerar o sinal.

Esse sinal então será gerado até que a função **noTone** seja chamada e pare de gerar a frequência no pino.

VARIAVEIS

Além da função **tone**, temos outra palavra nova em nosso código, a **int**.

Essa palavra está relacionada a um conceito muito importante, presente em praticamente todas as linguagens de programação: as **VARIÁVEIS**.

Imagine que você quer armazenar um valor numérico, o estado de um botão ou até mesmo a frequência com que o seu buzzer irá oscilar.

VARIAVEIS

As variáveis nos **permitem armazenar esses valores na memória de nossa placa** para que possamos recuperá-los em outro trecho do código.

Quando criamos uma variável, **estamos na verdade atribuindo um nome para uma seção da memória do microcontrolador**, e assim **aquele espaço** fica reservado para que possamos ocupá-lo com o valor que desejarmos.

FAZENDO BARULHO SIMPLES

VARIAVEIS

Uma boa analogia é você pensar que essa memória é como se fosse um **armário cheio de gavetas**, e que quando criamos uma variável, estamos na verdade rotulando uma delas para guardar um valor dentro dela.



FAZENDO BARULHO SIMPLES

VARIAVEIS

Para cada tipo e tamanho de conteúdo existe um tipo e tamanho de gaveta.

De acordo com o tipo do seu valor (**booleano**, **inteiro**, **racionais**, **etc**) e também do tamanho dele, **é necessário especificar um tipo de variável na hora de criá-la.**

FAZENDO BARULHO SIMPLES

VARIAVEIS

Por exemplo, se você está lendo o estado de um botão que só pode assumir **HIGH** ou **LOW** (que é o mesmo que **1** e **0** ou **verdadeiro** e **falso**), podemos utilizar uma variável do tipo **bool**.

Já para valores inteiros devemos utilizar o tipo **int**, ou **long** se o valor for realmente grande.

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

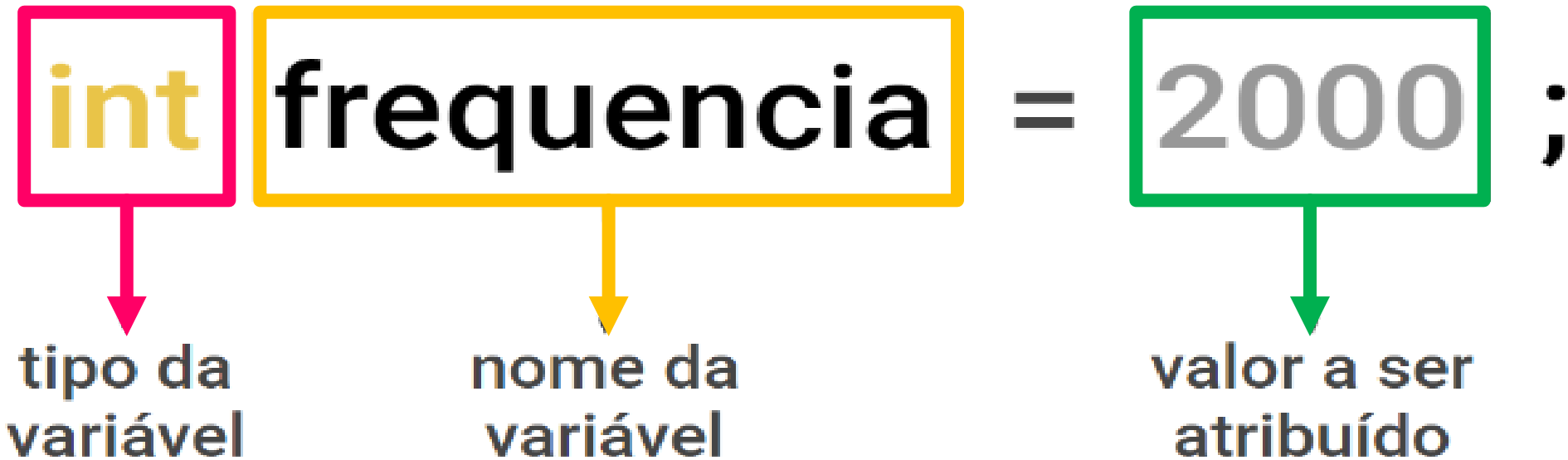
Nesse código criamos nossa primeira variável, a **frequencia**, responsável por armazenar o valor da frequência do sinal que será usado para acionar o buzzer.

Para declarar uma variável você precisa definir o **tipo**, o **nome** e opcionalmente deve **inicializá-la** com um valor válido.

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

Para declarar uma variável você precisa definir o **tipo**, o **nome** e opcionalmente deve **inicializá-la** com um valor válido.



FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

A primeira coisa a se fazer é definir o **tipo da variável** de acordo com o tipo de informação que você pretende salvar dentro dela. Abaixo você encontra uma tabela com esses tipos e respectivos tamanhos.

Tipo	Valores	Tamanho
bool	booleanos	HIGH ou LOW
byte	inteiros	0 a 255
char	inteiros	-128 a 127
int	inteiros	-32,768 a 32,767
long	inteiros	-2,147,483,648 a 2,147,483,647
float	reais	3.4028235E+38 a -3.4028235E+38

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

Em seguida você **determina o nome** que você precisará usar sempre que desejar usar ou alterar o valor que está armazenado dentro dela.

Para isso existem algumas regras que devem ser seguidas para que não aconteçam problemas de compilação ou de mau funcionamento:

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

1. O único caractere especial que pode ser utilizado é o ***underline*** ("_"). Letras acentuadas também são proibidas.
2. O nome da variável não pode ser iniciado com um número. Você pode usar números, mas **só após o primeiro caractere válido**.
3. Não pode haver espaços no nome da variável. Você pode separar palavras compostas com um ***underline***.
4. O nome de uma variável não pode ser igual ao de uma palavra reservada (setup, int, loop, etc).

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

Por último você atribui um valor inicial na variável para garantir que ela não esteja ocupada por sujeira da memória do microcontrolador, pois isso pode ser inconveniente.

Para atribuir esse valor fazemos uso de um **operador de atribuição**, representado por um sinal de igual ("=").

Este operador aritmético nos **permite modificar o conteúdo interno de uma variável.**

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

Os outros operadores permitem também que façamos operações matemáticas com os valores, como soma, multiplicação ou divisão.

Tem também o operador módulo (%) que resulta no resto da divisão de um número inteiro por outro.

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

Símbolo	Operação	Exemplo
=	atribuição	$a = b$
+	soma	$a + b$
-	subtração	$a - b$
*	multiplicação	$a * b$
/	divisão	a / b
%	módulo	$a \% b$

FAZENDO BARULHO SIMPLES

CRIANDO UMA VARIÁVEL

Quando utilizamos a instrução ***int frequencia = 2000***, estamos criando uma variável inteira cujo conteúdo será 2000.

Podemos então utilizá-la em outra parte do código, como na linha ***tone(10, frequencia)***.

Quando o compilador ler a palavra ***frequencia***, ele irá buscar o valor contido nela, que neste caso é **2000**.



PRATICANDO

FAZENDO FREQUENCÍMETRO

Para que possamos entender com clareza como a frequência influencia no som emitido pelo buzzer e como que as variáveis funcionam, vamos ampliar nosso circuito e montar um frequencímetro.

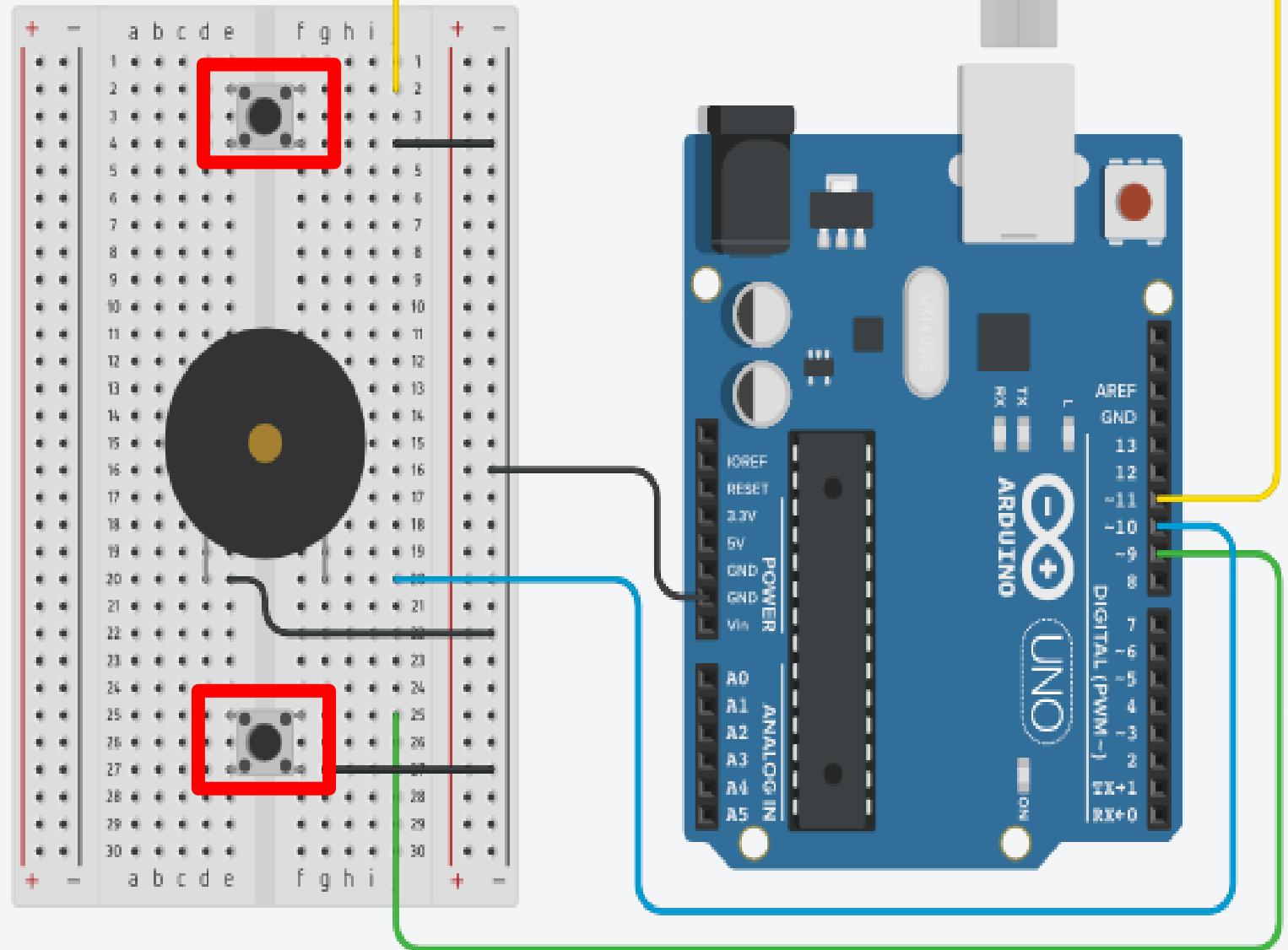
No próximo slide veremos um exemplo de como realizar essa ligação.

FAZENDO FREQUENCÍMETRO

CIRCUITO

Para isso vamos adicionar **dois botões** ao circuito que já está montado em nossa protoboard.

Nota: dessa vez não utilizaremos resistores junto aos nossos botões, pois vamos usar um recurso da nossa placa Arduino que permite ativar um resistor interno para desempenhar uma função similar.



CÓDIGO

Vamos programar a placa para que os botões consigam alterar a frequência do buzzer entre um acionamento e outro, permitindo estudar o comportamento em frequências altas e baixas.

FAZENDO FREQUENCÍMETRO

CÓDIGO

```
1. /*****
2. * Projeto Freqüencímetro
3. * Aumenta e diminui a frequência no buzzer de
   acordo com os botões.
4. *****/
5.
6.  int frequência = 2000; // frequência inicial do buzzer
7.  const int pinoBuzzer = 10; // pino onde o buzzer esta
   conectado
8.  const int pinoBotao1 = 9; // pino onde o botao 1 esta
   conectado
```

CONTINUA...

FAZENDO FREQUENCÍMETRO

...CONTINUAÇÃO

```
9.  const int pinoBotao2 = 11;  // pino onde o botao 2 esta
    conectado
10.
11. void setup() {
12.   pinMode(pinoBuzzer, OUTPUT); // configura o pino com o
    buzzer como saída
13.   pinMode(pinoBotao1, INPUT_PULLUP); // configura o
    pino com o Botão como entrada
14.   pinMode(pinoBotao2, INPUT_PULLUP); // configura o
    pino com o Botão como entrada
15. }
16.
```

CONTINUA...

FAZENDO FREQUENCÍMETRO

...CONTINUAÇÃO

```
17. void loop() {
18.
19.     // Soma 100 à frequencia atual se o botao estiver pressionado
20.     if (digitalRead(pinoBotao1) == LOW) {
21.         frequencia = frequencia + 100;
22.     }
23.
24.     // Subtrai 100 da frequencia atual se o botao estiver pressionado
25.     if (digitalRead(pinoBotao2) == LOW) {
26.         frequencia = frequencia - 100;
27.     }
```

CONTINUA...

FAZENDO FREQUENCÍMETRO

...CONTINUAÇÃO

```
28.  
29.  tone(pinoBuzzer,  frequencia); // gera frequencia no  
    buzzer  
30.  delay(1000);  
31.  noTone(pinoBuzzer); // para frequencia no buzzer  
32.  delay(1000);  
33. }
```

FIM!

MONTANDO no ARDUINO

FAZENDO FREQUENCÍMETRO

Desconecte a placa de seu computador
e monte o circuito conforme montou no TINKERCAD.
Lembre-se do que falamos sobre os BUZZER para
conectá-lo corretamente em sua protobard.



FAZENDO FREQUENCÍMETRO

Antes de você apertar qualquer um dos botões, o comportamento do sistema deve ser o mesmo do início dessa aula, **um som na frequência de 2000 Hz a cada 1 segundo.**

Ao apertar e segurar o botão, você poderá ver que a frequência sonora será alterada, para mais ou para menos, dependendo de qual botão você mantiver pressionado.

FAZENDO FREQUENCÍMETRO

Na construção do código **temos algumas novidades**. Uma delas é a utilização de variáveis para armazenar o número do pino no qual estamos conectando nossos componentes.

Isso é uma boa prática por dois motivos básicos:

- facilidade na leitura do código, pois ao ler a variável você já sabe o que está conectado,
- e também na manutenção do código, pois se você precisar mudar um componente de pino, basta mudar apenas no valor atribuído na declaração da variável e não em todos os pontos onde o pino foi utilizado.

FAZENDO FREQUENCÍMETRO

Note que tem uma palavrinha **const** antes do **int**. Essa palavra indica para o compilador que o valor armazenado dentro da variável é constante e **não pode ser alterado ao longo da execução do código**.

Isso não é obrigatório, mas é uma boa prática, **pois estamos tratando de pinos que normalmente não mudarão durante a execução do programa**.

Você pode utilizar esse recurso sempre que quiser prevenir a mudança do valor de uma variável que deve permanecer fixo.

FAZENDO FREQUENCÍMETRO

Outra novidade é a utilização da palavra **INPUT_PULLUP** na configuração dos pinos dos botões como entrada.

Ao utilizar essa instrução, **não só estamos declarando o pino como entrada, mas também habilitando internamente no chip um resistor entre o pino e o VCC**, portanto, criando um resistor de pull-up em nosso botão.

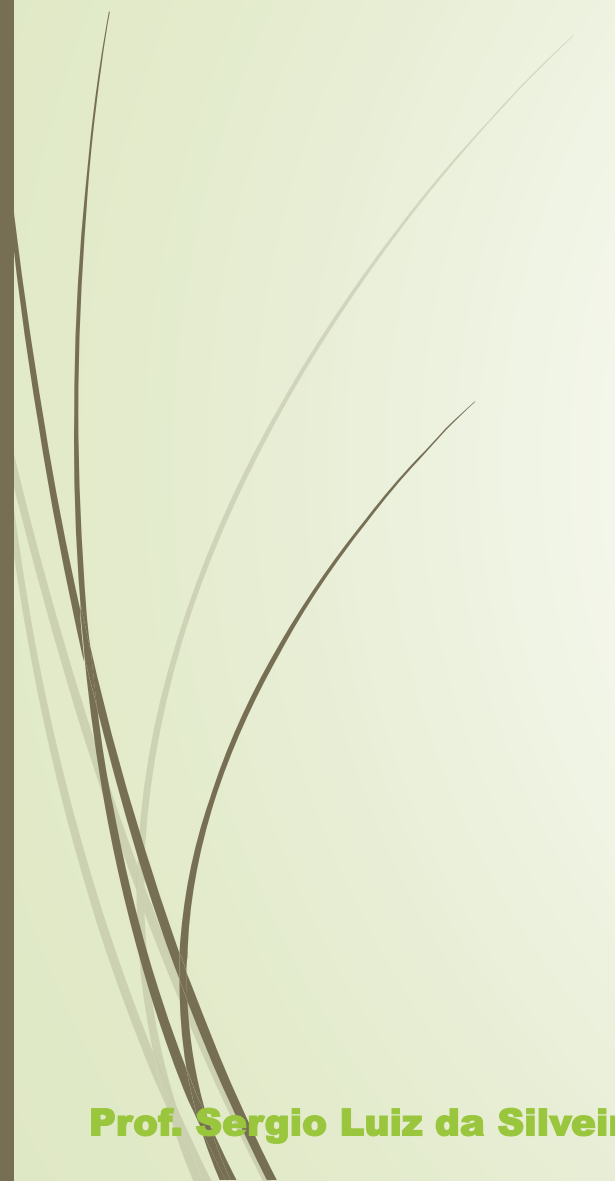
FAZENDO FREQUENCÍMETRO

A **função deste resistor** é a mesma que o resistor de pull-down, **mas** dessa vez, quando você pressiona o botão, o valor no pino é **LOW** e não **HIGH**.

Por este motivo é que na comparação temos

```
if (digitalRead(pinoBotao2) == LOW).
```

Fora isso, o resto você já deve conseguir entender a partir dos conceitos aprendidos até o momento.



FIM!